UNIT II: Functions and Classes & Objects in C++

Name: Kavitha B **Registration No:** 192565041

Section A: Debugging Questions

Question 1: Dangling Reference in Return-by-Reference

Defect classification

Line	Defect	Classification
4 to 8	return result; in getMaximumScaled() returns int& to the local variable result	Dangling reference / Undefined Behavior (lifetime error). g++: warning: reference to local variable 'result' returned [-Wreturn-local-addr]
19	int &danglingRef binds to the destroyed object	Dangling reference created
27	Reads danglingRef after other calls reused the stack	Undefined Behavior: garbage value or segmentation fault

When the buggy program was run it crashed with a segmentation fault (exit code 139). Another compiler or optimisation level may instead print a garbage value, or even 40 by luck; that unpredictability is the symptom of undefined behavior.

Task 1: What happens to result after the function ends? Why is Line 27 undefined?

result is an automatic (stack) variable that lives in the activation record of getMaximumScaled. Its storage duration ends at the closing brace of the function (C++ standard, [basic.stc.auto]), so the stack frame is popped and that memory is released for reuse. The reference returned to the caller still holds the old address, so danglingRef is an alias for an object whose lifetime has ended. Using a reference to an object outside its lifetime is undefined behavior ([basic.life]); the standard requires no diagnostic and permits any result.

Between Line 19 and Line 27 the program calls getArrayElement() and several std::cout operator<< functions. Each call pushes a new frame onto the same stack region and overwrites the old bytes where result used to be. So the read on Line 27 sees whatever data now sits at that address: a garbage number, or a crash.

Task 2: Why is getArrayElement safe but getMaximumScaled unsafe?

getArrayElement returns arr[index]. The parameter arr is a pointer to data[] declared in main(), so the element belongs to the caller and lives until main() ends. The returned reference therefore outlives the call, and the lvalue assignment getArrayElement(data, 1) = 999 modifies the real array element. getMaximumScaled returns a reference to an object that is created and destroyed inside the call.

Rule: return by reference only an object that outlives the function: a caller-owned object (reference or pointer parameter), a static or global object, heap memory, or a data member of *this. Never return a reference to a local automatic variable or a by-value parameter.

Task 3: Corrected code (return by value)

```
#include <iostream>

// Returns the scaled maximum BY VALUE: the caller gets its own copy.
int getMaximumScaled(int a, int b, int scaleFactor) {
    int maxVal = (a > b) ? a : b;
    int result = maxVal * scaleFactor;
    return result;                  // copy of result is returned (safe)
```

```
}

// Safe return-by-reference: arr[index] lives in the caller's array.
int& getArrayElement(int arr[], int index) {
    return arr[index];
}

int main() {
    int x = 10, y = 20;

    int scaled = getMaximumScaled(x, y, 2);   // own copy, not a reference
    std::cout << "Scaled Max: " << scaled << std::endl;

    int data[3] = {100, 200, 300};
    getArrayElement(data, 1) = 999;           // lvalue assignment
    std::cout << "Updated Array[1]: " << data[1] << std::endl;

    std::cout << "Scaled Max (still valid): " << scaled << std::endl;
    return 0;
}
```

getMaximumScaled now returns int by value, so the caller receives its own copy and the local variable can die safely. getArrayElement keeps returning int&, which is correct because the element belongs to the caller.

Task 4: Dry run and expected output

Step	Statement executed	State / action
1	int x = 10, y = 20;	x = 10, y = 20
2	scaled = getMaximumScaled(x, y, 2)	a = 10, b = 20, scaleFactor = 2; maxVal = 20; result = 40; a copy of 40 is returned; frame destroyed
3	cout << "Scaled Max: " << scaled	scaled = 40, so it prints Scaled Max: 40
4	int data[3] = {100, 200, 300};	data = {100, 200, 300}
5	getArrayElement(data, 1) = 999;	returns reference to data[1], then 999 is stored: data = {100, 999, 300}
6	cout << data[1]	prints Updated Array[1]: 999
7	cout << scaled	scaled is an independent copy, still 40, so it prints Scaled Max (still valid): 40

Console output

```
Scaled Max: 40
Updated Array[1]: 999
Scaled Max (still valid): 40
```

Question 2: Static Data Member Linkage & Static Member Function Scoping

Defect classification

Line	Defect	Classification
19 (g++ line 20)	Static displayAudit() prints accHolder, a non-static member	Compilation error. g++: invalid use of member 'BankAccount::accHolder' in static member function
8	static double totalBankReserves; is declared but never defined at file scope	Linker error. ld: undefined reference to 'BankAccount::totalBankReserves' (reported for the constructor and for displayAudit)

Order of errors: the compilation error stops the build first. The linker error appears only after the compile error is fixed (checked by removing the accHolder line and rebuilding).

Task 1: The linker error

Inside the class, static double totalBankReserves; is only a declaration ([class.static.data]: the declaration of a static data member in its class definition is not a definition). A class definition describes the layout of each object, and a static member is not part of any object. It is one shared variable that must have exactly one definition in one translation unit (One Definition Rule), outside the class, written as double BankAccount::totalBankReserves = 0.0;. Without that line no storage is ever allocated, so the compiler happily compiles the uses in the constructor and in displayAudit, but the linker finds no symbol to bind them to and reports undefined reference. (Since C++17, declaring it inline static is an alternative that needs no external definition.)

Task 2: The compilation error on Line 19

A static member function belongs to the class, not to an object, so it has no this pointer ([class.static.mfct]). The expression accHolder really means this->accHolder, and without an object there is no way to know whose accHolder is meant: Alice's or Bob's. A static function may use only static data, other static functions, and non-static members through an explicit object (for example an object passed as a parameter).

Task 3: Corrected code

```
#include <iostream>
#include <string>

class BankAccount {
private:
    std::string accHolder;
    double balance;
    static double totalBankReserves;   // declaration only

public:
    BankAccount(std::string name, double initialDeposit) {
        accHolder = name;
        balance = initialDeposit;
        totalBankReserves += initialDeposit;
    }

    // Static: touches only static data
    static void displayAudit() {
        std::cout << "Total Reserves in Vault: $" << totalBankReserves << std::endl;
    }

    // Non-static: needs an object, may touch instance data
    void displayAccount() const {
        std::cout << "Account Holder: " << accHolder
                  << ", Balance: $" << balance << std::endl;
    }
};

// Definition + initialization at file scope (allocates the storage)
double BankAccount::totalBankReserves = 0.0;

int main() {
    BankAccount acc1("Alice", 5000.0);
    BankAccount acc2("Bob", 3000.0);

    acc1.displayAccount();
    acc2.displayAccount();
    BankAccount::displayAudit();
    return 0;
}
```

The audit is split into two methods: static displayAudit() for class-level data, and non-static displayAccount() for per-object data.

Task 4: Dry run and output

Step	Statement	totalBankReserves	Output
0	Program start: file-scope definition runs	0.0	
1	BankAccount acc1("Alice", 5000.0)	0 + 5000 = 5000	
2	BankAccount acc2("Bob", 3000.0)	5000 + 3000 = 8000	
3	acc1.displayAccount()	8000	Account Holder: Alice, Balance: $5000
4	acc2.displayAccount()	8000	Account Holder: Bob, Balance: $3000
5	BankAccount::displayAudit()	8000	Total Reserves in Vault: $8000

Console output

```
Account Holder: Alice, Balance: $5000
Account Holder: Bob, Balance: $3000
Total Reserves in Vault: $8000
```

Question 3: Function Overload Ambiguity via Default Arguments

Defect classification

Line	Defect	Classification
19 (g++ line 20)	computeVolume(5.0) matches Overload 1 and Overload 2 equally well	Compilation error: call of overloaded 'computeVolume(double)' is ambiguous
22 (g++ line 23)	computeVolume(3.0, 7.0) matches Overload 1 and Overload 3 equally well	Compilation error: call of overloaded 'computeVolume(double, double)' is ambiguous

Task 1: Why is Line 19 ambiguous?

Overload resolution ([over.match], [over.match.best]) works in three steps:

1. **Candidates:** all three functions named computeVolume.
2. **Viable functions:** a function is viable if the arguments can fill its parameters, and default arguments count as filling the missing ones. For one argument, Overload 1 (length, width = 1.0, height = 1.0) is viable because width and height have defaults, and Overload 2 (side) is viable as an exact fit. Overload 3 (radius, height) needs two arguments and has no default, so it is not viable.
3. **Best viable function:** the compiler compares the conversion of each argument. Here double to double is an identity conversion for both, so neither is better. Default arguments take no part in ranking, and neither function is a template or more specialised, so there is no tie-breaker. No single best function exists, so the call is ill-formed.

The conflicting signatures are Overload 1 computeVolume(double, double = 1.0, double = 1.0) and Overload 2 computeVolume(double). Each is separately legal; the error is raised only at the call.

Task 2: Does Line 22, computeVolume(3.0, 7.0), also conflict?

Yes. With two arguments, Overload 1 is viable (length = 3.0, width = 7.0, height takes its default 1.0) and Overload 3 is viable (radius = 3.0, height = 7.0). Overload 2 is not viable because it takes exactly one parameter. Both viable functions have exact matches (identity conversions) for both arguments, so again there is no best function and

the compiler reports ambiguity. The question paper only flags Line 19, but g++ rejects both lines (confirmed), so Line 22 would still fail after fixing Line 19 alone. The cause is that the defaults of Overload 1 make it overlap with every other overload that has 1 or 2 parameters.

Task 3 and 4: Refactor, corrected code and output

Remove the default arguments and give each overload a different number of parameters, so every call has exactly one viable candidate: 1 argument is a cube, 2 arguments is a cylinder (radius, height), 3 arguments is a cuboid (length, width, height). (If a cuboid with fewer arguments is still wanted, give it its own name, such as cuboidVolume(l, w = 1.0, h = 1.0), which cannot clash with anything.)

```
#include <iostream>

// 1 argument  -> cube
double computeVolume(double side) {
    return side * side * side;
}

// 2 arguments -> cylinder (radius, height)
double computeVolume(double radius, double height) {
    return 3.14159 * radius * radius * height;
}

// 3 arguments -> cuboid (no default arguments, so no overlap)
double computeVolume(double length, double width, double height) {
    return length * width * height;
}

int main() {
    std::cout << "Cube: "     << computeVolume(5.0)          << std::endl;
    std::cout << "Cylinder: " << computeVolume(3.0, 7.0)     << std::endl;
    std::cout << "Cuboid: "   << computeVolume(2.0, 3.0, 4.0) << std::endl;
    return 0;
}
```

Call	Viable overloads	Chosen / result
computeVolume(5.0)	only the 1-parameter one	cube: 5 x 5 x 5 = 125
computeVolume(3.0, 7.0)	only the 2-parameter one	cylinder: 3.14159 x 3 x 3 x 7 = 197.92017, printed as 197.92 (cout default 6 significant digits)
computeVolume(2.0, 3.0, 4.0)	only the 3-parameter one	cuboid: 2 x 3 x 4 = 24

Console output

```
Cube: 125
Cylinder: 197.92
Cuboid: 24
```

Question 4: Array of Objects Missing Default Constructor

Defect classification

Line	Defect	Classification
20 (g++ line 23)	Student batch[3]; needs Student::Student() which does not exist	Compilation error: no matching function for call to 'Student::Student()'

Task 1: The rule and why the array fails

The compiler implicitly declares a default constructor only when the class has no user-declared constructor at all ([class.default.ctor]). Student declares Student(int, std::string), so the implicit Student() is suppressed. Declaring Student batch[3]; default-initialises each of the three elements ([dcl.init]), which calls a zero-argument constructor for every element. That constructor does not exist, so the declaration is ill-formed.

Task 2: Two techniques

- **Technique A: explicit default constructor.** Add Student() : rollNumber(0), name("Unknown") {} (or give every parameter of the existing constructor a default value). Student batch[3]; then compiles, and the elements can be filled afterwards.
- **Technique B: array initializer list.** Initialise each element with the parameterized constructor so no default constructor is required:

```
Student batch[3] = { Student(101, "Aarav"),
                     Student(102, "Bhavna"),
                     Student(103, "Chirag") };
```

This was compiled and gives the same output. Caveat: the list must supply an initializer for every element. If it had fewer than 3 entries, the remaining elements would need a default constructor again.

Task 3: Corrected program using Technique A

```
#include <iostream>
#include <string>

class Student {
private:
    int rollNumber;
    std::string name;

public:
    Student() : rollNumber(0), name("Unknown") {}          // Technique A: explicit default ctor
    Student(int r, std::string n) : rollNumber(r), name(n) {}

    void display() const {
        std::cout << "Roll: " << rollNumber << ", Name: " << name << std::endl;
    }
};

int main() {
    Student batch[3];                       // OK: uses Student()

    batch[0] = Student(101, "Aarav");
    batch[1] = Student(102, "Bhavna");
    batch[2] = Student(103, "Chirag");

    for (int i = 0; i < 3; ++i) {
        batch[i].display();
    }
    return 0;
}
```

Task 4: Dry run and verified output

Step	Statement	State of batch[0..2]
1	Student batch[3];	three calls to Student(): each is (0, "Unknown")
2	batch[0] = Student(101, "Aarav");	batch[0] = (101, Aarav)

Step	Statement	State of batch[0..2]
3	batch[1] = Student(102, "Bhavna");	batch[1] = (102, Bhavna)
4	batch[2] = Student(103, "Chirag");	batch[2] = (103, Chirag)
5	loop i = 0, 1, 2 calls display()	prints each record in order

Console output

```
Roll: 101, Name: Aarav
Roll: 102, Name: Bhavna
Roll: 103, Name: Chirag
```

Question 5: Friend Function Declaration & Scope Resolution Flaws

Defect classification

Line	Defect	Classification
4 (error at the friend declaration, line 15)	Forward-declares Engine, which is defined right after; TelemetryAuditor is used before it is declared	Compilation error: 'TelemetryAuditor' has not been declared
29 (g++ line 31)	alertThreshold used with no object	Compilation error: 'alertThreshold' was not declared in this scope
41 (g++ line 43)	v8.inspectDiagnostics(v8, audit) calls a friend as if it were a member	Compilation error: 'class Engine' has no member named 'inspectDiagnostics'
27	Objects passed by value (copies of both objects)	Inefficiency, not an error; fixed by passing const references

Task 1: Why alertThreshold cannot be used directly; does a friend have this?

A friend function is not a member of the class ([class.friend]). It has no implicit this pointer and is not in the scope of either class, so an unqualified name like alertThreshold is looked up as an ordinary identifier and is not found. Friendship grants only the right to access private and protected members; it does not bring them into scope. The access must be explicitly qualified by an object: a.alertThreshold, and likewise e.temperature and e.rpm.

Task 2: Why v8.inspectDiagnostics(v8, audit) fails

The dot operator looks the name up in the class of v8. The friend declaration inside Engine does not add a member, it only grants access to a non-member function. The correct call is an ordinary function call: inspectDiagnostics(v8, audit);.

Task 3: Why TelemetryAuditor needs a forward declaration before Engine

A name must be declared before it is used (point of declaration, [basic.scope.pdecl]). The friend declaration in Engine mentions TelemetryAuditor as a parameter type, but class TelemetryAuditor is defined only after Engine. The line class Engine; on Line 4 is useless, because Engine is defined immediately after. Writing class TelemetryAuditor; (an incomplete-type declaration) is enough for a function declaration to name the type; the function definition, placed after both full class definitions, then sees complete types and can access the members.

Task 4: Corrected code

```
#include <iostream>

class TelemetryAuditor;                    // forward declaration of the class used first

class Engine {
private:
```

```
    int temperature;
    int rpm;

public:
    Engine(int t, int r) : temperature(t), rpm(r) {}
    friend void inspectDiagnostics(const Engine& e, const TelemetryAuditor& a);
};

class TelemetryAuditor {
private:
    int alertThreshold;

public:
    TelemetryAuditor(int threshold) : alertThreshold(threshold) {}
    friend void inspectDiagnostics(const Engine& e, const TelemetryAuditor& a);
};

void inspectDiagnostics(const Engine& e, const TelemetryAuditor& a) {
    if (e.temperature > a.alertThreshold) {          // qualified with object names
        std::cout << "Warning: Engine overheat!" << std::endl;
    } else {
        std::cout << "Engine operating nominally at " << e.rpm << " RPM." << std::endl;
    }
}

int main() {
    Engine v8(105, 3200);
    TelemetryAuditor audit(100);

    inspectDiagnostics(v8, audit);                   // ordinary function call
    return 0;
}
```

Dry run and output

Step	Action	Value
1	Engine v8(105, 3200);	temperature = 105, rpm = 3200
2	TelemetryAuditor audit(100);	alertThreshold = 100
3	inspectDiagnostics(v8, audit);	e.temperature (105) > a.alertThreshold (100) is true
4	if-branch taken	prints the warning

Console output

```
Warning: Engine overheat!
```

If the engine temperature were 90 (not above 100), the else branch would print: Engine operating nominally at 3200 RPM.

Section B: Pseudo-Code Writing Questions

Question 1: Corporate Vault Management with Static Data & Functions

Task 1: Pseudo-code

```
CLASS BranchOffice
  PRIVATE:
    branchID            : STRING
    branchName          : STRING
```

```
    currentBalance      : REAL
    STATIC totalVaultReserves  : REAL    = 0.0     // shared by all branches
    STATIC totalBranchesActive : INTEGER = 0
  PUBLIC:
    CONSTRUCTOR(id : STRING, name : STRING, openingDeposit : REAL)
    METHOD Deposit(amount : REAL)
    METHOD Withdraw(amount : REAL) RETURNS BOOLEAN
    STATIC METHOD GetVaultSummary()
    STATIC METHOD GetGlobalLiquidityRatio(targetReserve : REAL) RETURNS REAL
ENDCLASS

// ----- one-time definition of the static attributes (outside the class) -----
REAL    BranchOffice::totalVaultReserves  <- 0.0
INTEGER BranchOffice::totalBranchesActive <- 0

CONSTRUCTOR BranchOffice::BranchOffice(id, name, openingDeposit)
  branchID       <- id
  branchName     <- name
  currentBalance <- openingDeposit
  totalVaultReserves  <- totalVaultReserves + openingDeposit
  totalBranchesActive <- totalBranchesActive + 1
END CONSTRUCTOR

METHOD BranchOffice::Deposit(amount)
  IF amount > 0 THEN
    currentBalance     <- currentBalance + amount
    totalVaultReserves <- totalVaultReserves + amount
  ENDIF
END METHOD

METHOD BranchOffice::Withdraw(amount) RETURNS BOOLEAN
  IF amount > 0 AND amount <= currentBalance THEN
    currentBalance     <- currentBalance - amount
    totalVaultReserves <- totalVaultReserves - amount
    RETURN TRUE
  ELSE
    RETURN FALSE
  ENDIF
END METHOD

STATIC METHOD BranchOffice::GetVaultSummary()
  // static: uses only static attributes, no instance data
  OUTPUT "Branches active : ", totalBranchesActive
  OUTPUT "Total reserves  : ", totalVaultReserves
END METHOD

STATIC METHOD BranchOffice::GetGlobalLiquidityRatio(targetReserve) RETURNS REAL
  IF targetReserve = 0 THEN
    OUTPUT "Error: target reserve cannot be zero"
    RETURN 0.0
  ENDIF
  RETURN totalVaultReserves / targetReserve
END METHOD

PROCEDURE Main
  b1 <- NEW BranchOffice("B001", "Chennai Central", 500000)
  b2 <- NEW BranchOffice("B002", "Madurai South",   300000)
  b1.Deposit(150000)
  ok <- b2.Withdraw(50000)
  OUTPUT "Withdrawal accepted: ", ok
  BranchOffice::GetVaultSummary()                       // class-scope call
  OUTPUT "Liquidity ratio: ", BranchOffice::GetGlobalLiquidityRatio(1000000)
END PROCEDURE
```

Task 2: Trace table

Assumed opening deposits (not given in the question): Branch1 = 500,000 and Branch2 = 300,000. The $150k deposit and $50k withdrawal are 150,000 and 50,000.

Step	Operation	b1.balance	b2.balance	totalVaultReserves	branches
0	Initial state (static definitions, no objects)	-	-	0.0	0
1	b1 = BranchOffice("B001", ..., 500000)	500,000	-	500,000	1
2	b2 = BranchOffice("B002", ..., 300000)	500,000	300,000	800,000	2
3	b1.Deposit(150000)	650,000	300,000	950,000	2
4	b2.Withdraw(50000): 50,000 <= 300,000, returns TRUE	650,000	250,000	900,000	2
5	GetVaultSummary()	650,000	250,000	900,000	2
6	GetGlobalLiquidityRatio(1,000,000) = 900,000 / 1,000,000	650,000	250,000	900,000	2

Output: Withdrawal accepted: TRUE; Branches active: 2; Total reserves: 900,000; Liquidity ratio: 0.9. The final static reserve is 900,000 (the 800,000 opening total plus 150,000 minus 50,000).

Question 2: Matrix Engine with Arrays in Class & Friend Dot-Product

Task 1: Pseudo-code

```
CLASS Matrix2D
  PRIVATE:
    grid : ARRAY[3][3] OF REAL
  PUBLIC:
    METHOD Initialize()
    METHOD SetElement(row : INTEGER, col : INTEGER, value : REAL)
    METHOD GetElement(row : INTEGER, col : INTEGER) RETURNS REAL
    METHOD Display()
    FRIEND FUNCTION ComputeCrossTrace(A : REF Matrix2D, B : REF Matrix2D) RETURNS REAL
ENDCLASS

// ----- member functions defined outside the class (scope resolution) -----
METHOD Matrix2D::Initialize()
  FOR i <- 0 TO 2
    FOR j <- 0 TO 2
      grid[i][j] <- 0.0
    NEXT j
  NEXT i
END METHOD

METHOD Matrix2D::SetElement(row, col, value)
  IF row < 0 OR row > 2 OR col < 0 OR col > 2 THEN
    OUTPUT "Error: index out of range"
    RETURN
  ENDIF
  grid[row][col] <- value
END METHOD

METHOD Matrix2D::GetElement(row, col) RETURNS REAL
  IF row < 0 OR row > 2 OR col < 0 OR col > 2 THEN
    OUTPUT "Error: index out of range"
    RETURN 0.0
  ENDIF
  RETURN grid[row][col]
```

```
END METHOD

METHOD Matrix2D::Display()
  FOR i <- 0 TO 2
    OUTPUT grid[i][0], grid[i][1], grid[i][2]       // one row per line
  NEXT i
END METHOD

// ----- friend function: NOT a member, no Matrix2D:: prefix, no "this" -----
FUNCTION ComputeCrossTrace(A : REF Matrix2D, B : REF Matrix2D) RETURNS REAL
  sum <- 0.0
  FOR i <- 0 TO 2
    sum <- sum + A.grid[i][i] * B.grid[i][i]    // direct private access
  NEXT i
  RETURN sum
END FUNCTION

PROCEDURE Main
  m1.Initialize()
  m2.Initialize()
  m1.SetElement(0,0,2)  m1.SetElement(1,1,4)  m1.SetElement(2,2,6)
  m2.SetElement(0,0,1)  m2.SetElement(1,1,3)  m2.SetElement(2,2,5)
  OUTPUT ComputeCrossTrace(m1, m2)               // plain call, not m1.Compute...
END PROCEDURE
```

The question writes the parameters by value. The pseudo-code passes them by REF to avoid copying 9-element arrays; the result is identical, and the friend does not modify them.

Task 2: Trace of ComputeCrossTrace (A diagonal [2, 4, 6], B diagonal [1, 3, 5])

i	A.grid[i][i]	B.grid[i][i]	Product	sum after step
init	-	-	-	0.0
0	2	1	2 x 1 = 2	2
1	4	3	4 x 3 = 12	14
2	6	5	6 x 5 = 30	44

Result: ComputeCrossTrace(A, B) = 44.

Question 3: Student Academic Roster via Array of Objects

Task 1: Pseudo-code

```
CLASS StudentRecord
  PRIVATE:
    rollNo      : INTEGER
    studentName : STRING
    marks       : ARRAY[3] OF REAL
    percentage  : REAL
    grade       : CHARACTER
    METHOD CalculateResult()                    // internal helper
  PUBLIC:
    CONSTRUCTOR()                               // default constructor
    METHOD ReadData(r : INTEGER, name : STRING, m1 : REAL, m2 : REAL, m3 : REAL)
    METHOD GetPercentage() RETURNS REAL
    METHOD Display()
ENDCLASS

CONSTRUCTOR StudentRecord::StudentRecord()
  rollNo <- 0
  studentName <- ""
  FOR i <- 0 TO 2 : marks[i] <- 0.0
```

```
  percentage <- 0.0
  grade <- ' '
END CONSTRUCTOR

METHOD StudentRecord::ReadData(r, name, m1, m2, m3)
  rollNo <- r
  studentName <- name
  marks[0] <- m1
  marks[1] <- m2
  marks[2] <- m3
  CalculateResult()
END METHOD

METHOD StudentRecord::CalculateResult()
  percentage <- (marks[0] + marks[1] + marks[2]) / 3
  IF      percentage >= 85 THEN grade <- 'A'
  ELSE IF percentage >= 70 THEN grade <- 'B'
  ELSE IF percentage >= 50 THEN grade <- 'C'
  ELSE                          grade <- 'F'
  ENDIF
END METHOD

METHOD StudentRecord::GetPercentage() RETURNS REAL
  RETURN percentage
END METHOD

METHOD StudentRecord::Display()
  OUTPUT rollNo, studentName, percentage, grade
END METHOD

PROCEDURE Main
  roster : ARRAY[3] OF StudentRecord            // default constructor runs 3 times
  roster[0].ReadData(101, "Bala",  60, 70, 65)
  roster[1].ReadData(102, "Asha",  90, 85, 80)
  roster[2].ReadData(103, "Charu", 75, 80, 70)

  n <- 3
  FOR i <- 0 TO n - 2                           // Bubble Sort, descending
    FOR j <- 0 TO n - 2 - i
      IF roster[j].GetPercentage() < roster[j+1].GetPercentage() THEN
        temp       <- roster[j]                // swap whole objects
        roster[j]  <- roster[j+1]
        roster[j+1] <- temp
      ENDIF
    NEXT j
  NEXT i

  OUTPUT "Rank  Roll  Name   %    Grade"
  FOR k <- 0 TO n - 1
    OUTPUT k + 1 ; roster[k].Display()
  NEXT k
END PROCEDURE
```

Task 2: Dry-run trace

Percentages: Bala (60 + 70 + 65) / 3 = 65.0 (grade C); Asha (90 + 85 + 80) / 3 = 85.0 (grade A); Charu (75 + 80 + 70) / 3 = 75.0 (grade B).

Pass	j	Compare (percentages)	Swap?	Array after step
-	-	Initial array	-	[Bala 65, Asha 85, Charu 75]
1	0	roster[0]=65 < roster[1]=85	YES	[Asha 85, Bala 65, Charu 75]
1	1	roster[1]=65 < roster[2]=75	YES	[Asha 85, Charu 75, Bala 65]

Pass	j	Compare (percentages)	Swap?	Array after step
2	0	roster[0]=85 < roster[1]=75 ? false	NO	[Asha 85, Charu 75, Bala 65]

Two passes (n - 1) and two swaps. The inner loop shrinks by one each pass because the smallest value has already reached the end.

Console output

```
Rank  Roll  Name   %      Grade
1     102   Asha   85.00  A
2     103   Charu  75.00  B
3     101   Bala   65.00  C
```

Question 4: Flexible Logger with Overloaded Functions & Inline Helpers

Task 1: Pseudo-code with signatures and inline definitions

```
INLINE FUNCTION FormatPrefix(prefixTag : STRING) RETURNS STRING
  RETURN "[" + prefixTag + "] "
END FUNCTION

// Overload 1: message only, default prefix INFO
FUNCTION LogEvent(message : STRING)
  OUTPUT FormatPrefix("INFO") + message
END FUNCTION

// Overload 2: severity (required) and timestamp (default)
FUNCTION LogEvent(message : STRING, severityLevel : INTEGER,
                  timestamp : STRING = "00:00:00")
  IF      severityLevel = 1 THEN name <- "LOW"
  ELSE IF severityLevel = 2 THEN name <- "MEDIUM"
  ELSE IF severityLevel = 3 THEN name <- "CRITICAL"
  ELSE                           name <- "UNKNOWN"
  ENDIF
  OUTPUT FormatPrefix(name) + "@ " + timestamp + " : " + message
END FUNCTION

// Overload 3: hardware fault
FUNCTION LogEvent(errorCode : INTEGER, systemModule : STRING)
  hex <- ConvertToHex(errorCode)                 // e.g. 255 -> "FF"
  OUTPUT FormatPrefix("HARDWARE_FAULT") + "Module: " + systemModule
         + " -> Error Code: 0x" + hex
END FUNCTION
```

Design note (ambiguity trap). If Overload 2 were given defaults for both severityLevel and timestamp, as the question words it, then LogEvent("text") would be viable for Overload 1 and for Overload 2 (both defaults used) with identical conversions, and the call would be ambiguous, exactly the trap seen in Debugging Question 3. To keep Overload 1 callable, severityLevel is therefore a required parameter above, and only timestamp has a default. (The other way out is to delete Overload 1 and let Overload 2 handle the one-argument case with both defaults.)

Task 2: Client calls and how they resolve

```
LogEvent("System boot")                      // 1 argument
LogEvent("Sensor drift", 2)                  // 2 arguments
LogEvent("Overheat", 3, "14:25:10")          // 3 arguments
LogEvent(255, "SPI_BUS")                     // 2 arguments, different types
```

Call	Viable overloads and chosen one	Default arguments used / output
LogEvent("System boot")	Overload 1 (1 arg). Overload 2 needs at least 2 args; Overload 3 needs 2. Chosen: 1	none [INFO] System boot
LogEvent("Sensor drift", 2)	Overload 2 (STRING, INTEGER). Overload 3 not viable: a STRING cannot convert to INTEGER errorCode. Chosen: 2	timestamp = "00:00:00" (default) [MEDIUM] @ 00:00:00 : Sensor drift
LogEvent("Overheat", 3, "14:25:10")	Only Overload 2 accepts 3 args. Chosen: 2	none, every argument supplied [CRITICAL] @ 14:25:10 : Overheat
LogEvent(255, "SPI_BUS")	Overload 3 (INTEGER, STRING). Overload 2 not viable: an INTEGER cannot convert to the STRING message. Chosen: 3	none; 255 converts to hex FF [HARDWARE_FAULT] Module: SPI_BUS -> Error Code: 0xFF

Default arguments are filled in by the compiler at the call site, only for trailing parameters, and only after the overload has been chosen by the number and types of the explicit arguments. Because FormatPrefix is INLINE, its body is expanded at each call site, which removes call overhead in high-throughput logging.

Question 5: Hardware Register Bank with Return-by-Reference

Task 1: Pseudo-code

```
CLASS RegisterBank
  PRIVATE:
    registers : ARRAY[8] OF INTEGER
    errorSink : INTEGER
  PUBLIC:
    CONSTRUCTOR()
    METHOD GetRegister(index : INTEGER) RETURNS REF INTEGER
    METHOD DisplayState()
ENDCLASS

CONSTRUCTOR RegisterBank::RegisterBank()
  FOR i <- 0 TO 7
    registers[i] <- 0
  NEXT i
  errorSink <- -1
END CONSTRUCTOR

METHOD RegisterBank::GetRegister(index) RETURNS REF INTEGER
  IF index >= 0 AND index < 8 THEN
    RETURN REF registers[index]          // alias of a member: outlives the call
  ELSE
    OUTPUT "ALERT: register index ", index, " out of range"
    RETURN REF errorSink                 // harmless sink, no illegal memory write
  ENDIF
END METHOD

METHOD RegisterBank::DisplayState()
  FOR i <- 0 TO 7
    OUTPUT "R", i, " = 0x", ConvertToHex(registers[i])
  NEXT i
END METHOD

PROCEDURE Main
  bank : RegisterBank
  bank.GetRegister(0) <- 170             // lvalue assignment: registers[0] = 170
  bank.GetRegister(1) <- 255             // registers[1] = 255
  readVal <- bank.GetRegister(0)         // read through reference: 170
```

```
  OUTPUT "readVal = ", readVal
  bank.GetRegister(10) <- 999            // out of range: alert, errorSink = 999
  bank.DisplayState()
END PROCEDURE
```

Client trace

Step	Statement	Target modified	State
0	bank constructed	-	registers[0..7] = 0; errorSink = -1
1	GetRegister(0) <- 170	registers[0]	registers[0] = 170 (0xAA)
2	GetRegister(1) <- 255	registers[1]	registers[1] = 255 (0xFF)
3	readVal <- GetRegister(0)	none (read)	readVal = 170
4	GetRegister(10) <- 999	errorSink (index invalid)	alert printed; errorSink = 999; registers unchanged
5	DisplayState()	-	R0 = 0xAA, R1 = 0xFF, R2 to R7 = 0x00

Task 2: How return-by-reference lets a call appear on the left of an assignment

An assignment needs a modifiable lvalue on its left side, that is, an expression that designates an existing object with an address. A function that returns by value produces a temporary copy (an rvalue), so func() = 5 would be rejected by the compiler, since a temporary cannot be assigned to.

When the return type is REF INTEGER (INTEGER& in C++), the function returns an alias to an object that already exists, not a copy. Behind the scenes the reference is a hidden address. By the rule that a call to a function returning an lvalue reference is itself an lvalue ([expr.call]), the call expression bank.GetRegister(0) designates registers[0]. The assignment <- 170 therefore stores 170 straight into registers[0]; no copy is involved, and the same alias reads the current value when it is used on the right side.

This is safe here because registers and errorSink are data members of bank, which outlive the call (see Debugging Question 1). Returning a reference to errorSink for a bad index guarantees the caller always receives a valid, writable object, so an out-of-range write cannot corrupt neighbouring memory.